**Колледж Научно-Технологического Университета Сириус**

---

# ПРАКТИЧЕСКАЯ РАБОТА

по дисциплине “Технология разработки программного обеспечения”

Выполнил:
Студент группы
Группы: k07090-23/1
Аведисян Артём Артурович
Принял:
Целищев Егор Дмитриевич

_____

IT-Колледж “Сириус”
2026

## Описание микросервисов

В системе реализовано 4 микросервиса, каждый из которых отвечает за отдельную часть функциональности приложения.

### 1. User & Profile Service

Отвечает за работу с пользователями и их анкетами. Обрабатывает регистрацию через Telegram ID, хранит личные данные пользователя (имя, возраст, пол, описание, геолокацию), управляет фотографиями и предпочтениями для поиска (возраст, пол, радиус). Реализуется с использованием Python + FastAPI для API, PostgreSQL для хранения данных, SQLAlchemy/Alembic для работы с БД и миграциями, Pydantic для валидации данных, а фотографии хранятся в Minio (S3).

### 2. Matching Service

Отвечает за логику знакомств. Формирует список анкет с учётом предпочтений пользователя и рейтинга других пользователей, обрабатывает лайки и пропуски, создаёт мэтчи при взаимной симпатии. Использует Redis для кэширования ленты анкет и RabbitMQ/Kafka для отправки событий (лайки, мэтчи) в другие сервисы. Реализуется на Python + FastAPI.

### 3. Chat Service

Отвечает за обмен сообщениями между пользователями. Обеспечивает отправку сообщений и хранение истории переписки после создания мэтча, а также получение списка сообщений по конкретному диалогу. Реализуется на Python + FastAPI, хранение сообщений — PostgreSQL, взаимодействие с другими сервисами через RabbitMQ, а фоновые задачи при необходимости через Celery.

### 4. Rating Service

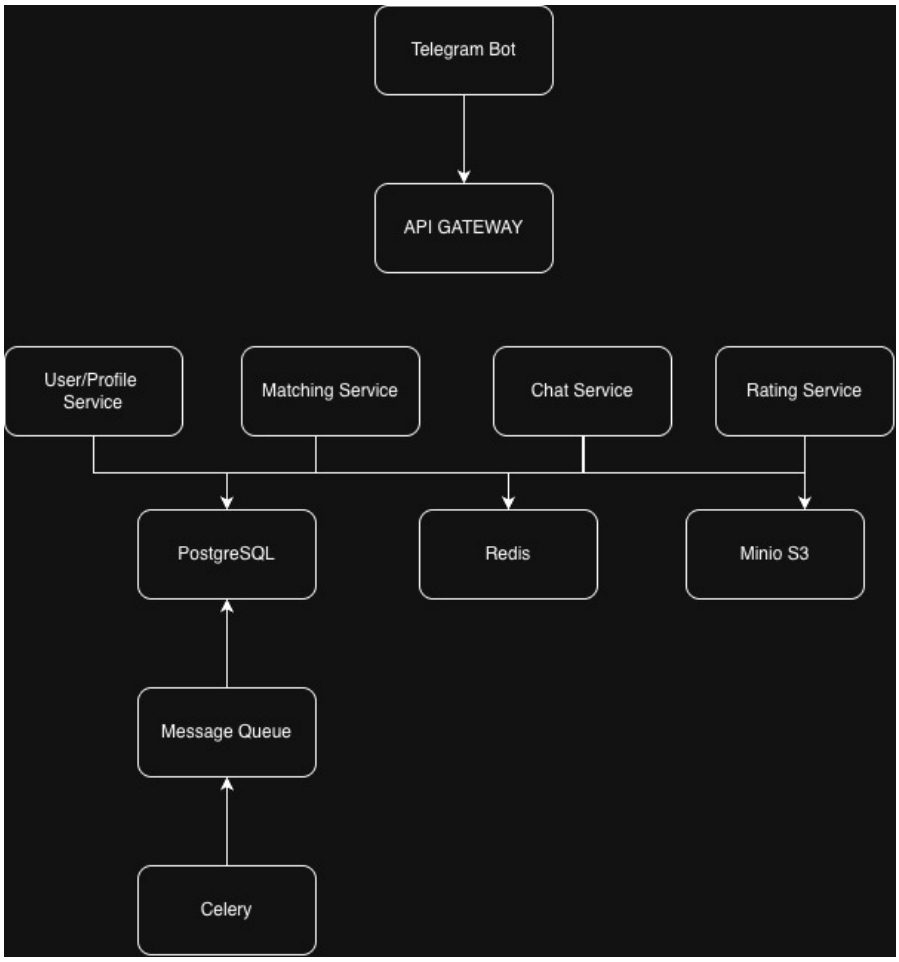
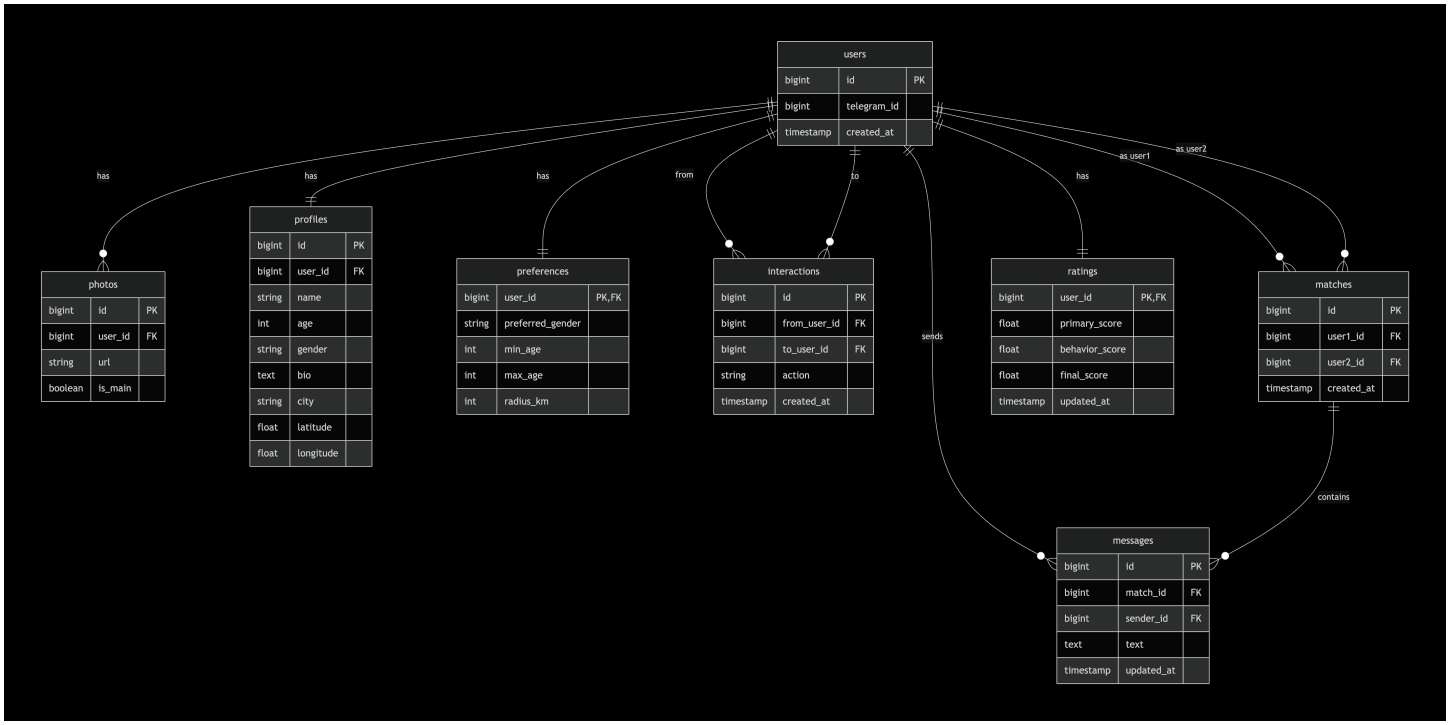
Отвечает за расчёт рейтинга пользователей. Реализует трёхуровневую систему ранжирования (первичный, поведенческий, итоговый рейтинг), хранит результаты в базе данных и обновляет их с помощью фоновых задач (Celery). Получает события (лайки, мэтчи, приглашения друзей) через RabbitMQ/Kafka и учитывает активность пользователей и реферальную систему. Реализуется на Python + FastAPI, хранение рейтингов — PostgreSQL, валидация данных через Pydantic.

## Дополнительно используемые компоненты

- **Redis** — кэширование анкет для ускорения выдачи
- **Message Queue (RabbitMQ / Kafka)** — обмен событиями между сервисами
- **Celery** — выполнение фоновых задач (пересчёт рейтингов)
- **Minio (S3)** — хранение фотографий

## Схема архитектуры

Схема показывает, как пользователь через Telegram-бота отправляет запросы в систему через API Gateway, после чего они распределяются между микросервисами (User/Profile, Matching, Chat, Rating). Сервисы обмениваются данными через API и очередь сообщений, используют общую базу данных, кэш Redis для ускорения работы и хранилище Minio для изображений, а фоновые задачи выполняются через Celery.



# Схема данных в БД